CHƯƠNG 1 · BÀI 2

1.2. Biến, kiểu dữ liệu & toán tử

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

Mục tiêu bài học

- Khai báo biến bằng var, let, const và hiểu sự khác nhau
- Nắm các kiểu dữ liệu của JavaScript
- Sử dụng toán tử và hiểu sự khác nhau giữa == và ===

1. Khai báo biến

ES6 bổ sung `let` và `const`. `const` dùng để khai báo biến không đổi (không gán lại được); `let` dùng cho biến có thể thay đổi và có phạm vi khối. `var` là cách cũ, có phạm vi hàm và dễ gây lỗi hoisting. Nên ưu tiên `const`, chỉ dùng `let` khi cần gán lại.

```
const PI = 3.14;
let dem = 0;
dem = dem + 1;
```

2. Kiểu dữ liệu

JavaScript có các kiểu nguyên thuỷ: number, string, boolean, null, undefined, symbol, bigint. Ngoài ra có object (gồm mảng và hàm). Một điểm gây nhầm lẫn nổi tiếng: `typeof null` trả về "object" (lỗi lịch sử của ngôn ngữ, vẫn được giữ để tương thích ngược).

```
console.log(typeof 42);        // "number"
console.log(typeof "abc");     // "string"
console.log(typeof undefined); // "undefined"
```

3. Toán tử

Toán tử số học giống nhiều ngôn ngữ: `+ - * / % **`. Toán tử + với chuỗi sẽ nối chuỗi. JavaScript có hai kiểu so sánh bằng: == so sánh sau khi ép kiểu, còn === so sánh cả giá trị lẫn kiểu. Nên luôn dùng === và !==.

```
console.log(5 == "5");   // true
console.log(5 === "5");  // false
console.log("5" + 3);    // "53"
console.log("5" - 3);    // 2
```

4. Giá trị đúng sai (truthy, falsy)

Các giá trị falsy gồm `false`, `0`, `""`, `null`, `undefined`, `NaN`; mọi giá trị khác là truthy. Toán tử logic là `&&`, `||`, `!`.

Tóm tắt

- Ưu tiên `const`, dùng `let` khi cần gán lại, tránh `var`.
- Kiểu nguyên thuỷ: number, string, boolean, null, undefined, symbol, bigint.
- Dùng `===` thay cho `==` để tránh ép kiểu bất ngờ.

Câu hỏi ôn tập và bài tập vận dụng

1. Khai báo các biến lưu tên, tuổi, trạng thái học viên và in kiểu của từng biến bằng typeof.
2. Dự đoán kết quả của `"10" * 2` và `"10" + 2`, sau đó chạy để kiểm tra.
3. Giải thích sự khác nhau giữa `==` và `===` kèm ví dụ.

Biến, kiểu dữ liệu & toán tử — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Hỏi thực tế: "const rồi sao vẫn push được?" const giữ binding không được gán lại, không đóng băng đối tượng. let cho phép gán lại; ưu tiên const khi không cần gán lại biến.

```
const items = [];
items.push("A");
console.assert(items.length === 1);
console.assert("2" + 3 === "23");
console.assert(Number("2") + 3 === 5);
console.assert(("2" == 2) === true);
console.assert(("2" === 2) === false);
```

Toán tử + có chuỗi sẽ nối chuỗi trong trường hợp trên. `==` có ép kiểu; `===` kiểm tra nghiêm ngặt, nên dùng khi không chủ đích ép kiểu. `Number("")` cho 0 nên kiểm tra rỗng trước khi đổi số.

Tự luyện: kiểm tra chuỗi khoảng trắng và `"abc"`. **Hướng giải:** loại input rỗng; sau chuyển đổi dùng `Number.isFinite`. Không dùng kiểm tra truthy để loại số 0 nếu 0 là dữ liệu hợp lệ.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide